

Python Data Visualization

Matplotlib & Seaborn — Complete Notebook Guide

Covers 4 notebooks:

1. [IntroToMatplotlib.ipynb](#) | 2. [MatplotlibStyles.ipynb](#)
3. [BoxPlotExercise.ipynb](#) | 4. [VisualizationSeaborn.ipynb](#)

Notebook 1: Intro to Matplotlib

File: `IntroToMatplotlib.ipynb`

Matplotlib is Python's foundational plotting library. This notebook teaches you to create and customize line plots from raw Python lists and NumPy arrays, and introduces the full figure/axes structure, subplots, and legends.

Setup & Installation

```
#! pip install matplotlib      # install if needed
```

```
import numpy as np
import matplotlib.pyplot as plt
```

```
%matplotlib inline    # displays plots inside Jupyter cells
```

The `%matplotlib inline` magic command is specific to Jupyter notebooks — it tells the notebook to render plots directly below the code cell rather than in a separate window.

Basic Line Plot — `plt.plot()`

Definition: Draws a 2D line connecting (x, y) data points. Accepts Python lists or NumPy arrays as inputs. The optional third argument is a format string that controls color and line style.

```
age_list    = [10, 20, 30, 30, 30, 40, 50, 60, 70, 75]
weight_list = [20, 60, 80, 85, 86, 87, 70, 90, 95, 90]

plt.plot(age_list, weight_list, "b")  # "b" = blue line
plt.xlabel("Age")
plt.ylabel("Weight")
plt.title("Age vs Weight")
plt.show()
```

Format string color codes

Property / Value	Description
"b"	Blue line
"g"	Green line
"r"	Red line
"y"	Yellow line
"b*"	Blue star markers (no line)
"b- -"	Blue dashed line
"b- *"	Blue solid line with star markers
"b* -"	Same as above — line with star markers
"r- -"	Red dashed line

Display and Save — `plt.show()` / `plt.savefig()`

`plt.show()` renders the current figure to the screen. `plt.savefig()` writes it to disk as a PNG, JPG, or PDF file.

```
plt.show()          # display the figure in the notebook
plt.savefig("test.png") # save as an image file on disk
```

NumPy Arrays — `np.array()` and `np.linspace()`

`np.array()` converts a Python list into a NumPy array, enabling fast mathematical operations on all elements at once.

```
np_age_list    = np.array(age_list)
np_weight_list = np.array(weight_list)

# Plot with green line – same syntax, but NumPy arrays underneath
```

```
plt.plot(np_age_list, np_weight_list, "g")
plt.show()
```

`np.linspace(start, stop, num)` — generates N evenly-spaced numbers between start and stop. Perfect for creating smooth x-axis data for mathematical functions.

```
numpy_arr1 = np.linspace(0, 10, 20)    # 20 numbers evenly from 0 to 10
numpy_arr2 = numpy_arr1 ** 3           # cube every element

plt.plot(numpy_arr1, numpy_arr2)
plt.show()
```

Line style format string combinations

Property / Value	Description
"b*"	Blue star markers only (no connecting line)
"b*-"	Blue line connecting star markers
"b--"	Blue dashed line
"b-*"	Blue solid line with star markers at each point

Subplots — `plt.subplot()`

Definition: Divides the figure into a grid of rows × columns and selects which cell to draw into. Each call to `plt.subplot()` activates a different cell.

```
plt.subplot(1, 2, 1)    # 1 row, 2 columns, activate 1st cell
plt.plot(numpy_arr1, numpy_arr2, "b-*")

plt.subplot(1, 2, 2)    # same grid, activate 2nd cell
plt.plot(numpy_arr2, numpy_arr1, "r--")
plt.show()
```

Property / Value	Description
nrows	Number of rows in the grid
ncols	Number of columns in the grid
index	Position to activate — 1-based, counting left to right, top to bottom

Manual Axis Positioning — `figure.add_axes()`

Definition: Adds a plot area at a precise position and size using normalized coordinates [left, bottom, width, height] — each value from 0.0 to 1.0 relative to the figure canvas.

```
my_figure = plt.figure()
```

```
figure_axes = my_figure.add_axes([0.1, 0.1, 0.5, 0.5])
                                # left bot width height
figure_axes.plot(numpy_arr1, numpy_arr2, "g")
figure_axes.set_xlabel("X Axis")
figure_axes.set_ylabel("Y Axis")
figure_axes.set_title("Graph Title")
plt.show()
```

Legend — axes.legend()

Definition: Displays a legend identifying each plotted line. Labels must be set with label= inside each plot() call.

```
new_fig = plt.figure(dpi=100) # dpi=100 → higher resolution
new_axes = new_fig.add_axes([0.1, 0.1, 0.9, 0.9])

new_axes.plot(numpy_arr1, numpy_arr1**2, label="arr ** 2")
new_axes.plot(numpy_arr1, numpy_arr1**3, label="arr ** 3")
new_axes.legend(loc=0) # loc=0 = best auto position
plt.show()
```

Property / Value	Description
loc=0	Best automatic position (avoids the data)
loc=1	Upper right corner
loc=2	Upper left corner
loc=3	Lower left corner
loc=4	Lower right corner
dpi	Dots per inch on the figure — higher value = sharper, larger output

Notebook 2: Matplotlib Styles

File: MatplotlibStyles.ipynb

This notebook builds on the intro by switching to the object-oriented `plt.subplots()` style and covering fine-grained line styling — hex colors, transparency, line widths, dash patterns, markers, and their fill colors. It also introduces scatter plots, histograms, and box plots.

Object-Oriented Approach — plt.subplots()

Definition: Returns a (Figure, Axes) tuple in one call. This is the modern, recommended way to create Matplotlib plots — it gives you direct named references to both objects.

```
my_fig, my_axes = plt.subplots()
# my_fig → matplotlib.figure.Figure
# my_axes → matplotlib.axes.Axes

my_axes.plot(data1, data2)
my_axes.plot(data2, data1)
plt.show()
```

Color — color parameter

Definition: Controls the color of a plotted line or element. Accepts hex codes like "#1E0703" for precise colors, named strings like "red", or single-letter shorthands.

```
my_axes.plot(data1, data2, color="#1E0703")
my_axes.plot(data2, data1, color="#53CA0A")
```

Transparency — alpha parameter

Definition: Controls how transparent a line or element is. Ranges from 0.0 (fully invisible) to 1.0 (fully opaque). Useful for overlapping lines.

```
my_axes.plot(data1, data2, color="#1E0703", alpha=0.5) # 50% transparent
```

Line Width — linewidth parameter

Definition: Controls the thickness of the plotted line in points.

```
new_axes.plot(data1, data1+2, color="#1E0703", linewidth=1.0) # thin
new_axes.plot(data1, data1+3, color="#53CA0A", linewidth=5.0) # thick
```

Line Style — linestyle parameter

Definition: Sets the dash pattern of the line.

```
new_axes.plot(data1, data1+5, color="#c85d09", linestyle="-.") # dash-dot
```

Property / Value	Description
"_"	Solid line (default)
"--"	Dashed line
"-."	Dash-dot alternating pattern
": "	Dotted line

Markers — marker, markersize, markerfacecolor

Definition: A marker is a symbol drawn at each data point. `markersize` controls its size; `markerfacecolor` fills the inside of closed markers (like circles).

```

new_axes.plot(
    data1, data1+7,
    color="#000000",
    linestyle="-. ",
    marker="o",           # circle at each data point
    markersize=8,
    markerfacecolor="#53CA0A" # green fill inside the circle
)

new_axes.plot(
    data1, data1+9,
    color="#000000",
    linestyle="--",
    marker="+",           # plus/cross markers
    markersize=4
)

```

Property / Value	Description
"o"	Circle marker
"+"	Plus / cross marker
"*"	Star marker
"s"	Square marker
"^"	Triangle marker (pointing up)
"v"	Triangle marker (pointing down)
markersize	Size of the marker symbol in points
markerfacecolor	Fill color inside closed markers (circles, squares, etc.)

Scatter Plot — plt.scatter()

Definition: Plots individual (x, y) points as dots without connecting lines. Best for showing correlation or distribution between two numeric variables.

```

plt.scatter(data1, data2)
plt.show()

```

Histogram — plt.hist()

Definition: Counts how many values fall into each equal-width range (bin) and draws vertical bars. Used to see the distribution and frequency of a single variable.

```

new_arr = np.random.randint(0, 100, 50) # 50 random integers 0-99
plt.hist(new_arr)
plt.show()

```

Box Plot (Matplotlib) — `plt.boxplot()`

Definition: Draws a statistical box plot showing the median, quartiles (Q1/Q3), whiskers ($1.5 \times \text{IQR}$), and outlier points. Particularly useful for detecting extreme values.

```
plt.boxplot(new_arr)    # good for identifying outliers
plt.show()
```

Notebook 3: Box Plot Exercise

File: `BoxPlotExercise.ipynb`

A focused, hands-on notebook that deeply explains every component of a box plot using Seaborn. Three carefully designed experiments show how different datasets change the box shape — including how outliers stretch the whiskers and how repeated values shift the median.

What is a Box Plot?

A box plot (also called a box-and-whisker plot) is a standardized visual summary of a dataset. It compresses an entire distribution into five key statistics, making it easy to compare datasets and spot unusual values at a glance.

Anatomy of a Box Plot

Property / Value	Description
Median (Q2 / 50%)	The middle value of the sorted dataset — drawn as the thick horizontal line inside the box
Q1 (lower quartile / 25%)	25% of all values fall below this line — the bottom edge of the box
Q3 (upper quartile / 75%)	75% of all values fall below this line — the top edge of the box
IQR (Interquartile Range)	$Q3 - Q1$. Represents the spread of the middle 50% of the data. The height of the box equals the IQR.
Whiskers	Lines extending above and below the box — they reach up to $1.5 \times \text{IQR}$ from the box edges. The notebook comment notes: 'the lines up and bottom is known as whiskers'
Outliers	Individual dots beyond the whiskers — data points more than $1.5 \times \text{IQR}$ away from Q1 or Q3

`sns.boxplot()` — Seaborn Box Plot

Definition: Draws a statistical box plot using Seaborn's styling engine, which produces more visually polished results than Matplotlib's `plt.boxplot()`. The `y=` parameter plots a vertical box for a single 1D dataset.

```
import numpy as np
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

data = [5, 7, 9, 15, 20, 22, 25, 30, 32, 35, 37, 40, 50, 55, 60, 100]

plt.figure(figsize=(6, 5)) # width=6in, height=5in
sns.boxplot(y=data)
plt.title("Box Plot")
plt.ylabel("Data Value")
plt.show()
```

Property / Value	Description
y=data	Single list/array → draws one vertical box plot
x=data	Single list/array → draws one horizontal box plot
figsize=(w, h)	Figure size in inches — set on plt.figure() before calling sns.boxplot()

Three Experiments

Experiment 1 — Normal dataset with mild outlier

A clean range of 16 values from 5 to 100. The value 100 is noticeably higher than the others and appears as an outlier dot above the top whisker.

```
data = [5, 7, 9, 15, 20, 22, 25, 30, 32, 35, 37, 40, 50, 55, 60, 100]
# 100 is an outlier – it plots as a dot above the top whisker
```

Experiment 2 — Extreme outlier (2000)

Adding the value 2000 dramatically changes the plot. Because 2000 is so far from the rest of the data, the box itself shrinks to a tiny band near the bottom of the chart and the whisker extends enormously. The notebook comment notes: "we can remove the outlier from our data set" — demonstrating why detecting and removing extreme outliers matters before analysis.

```
data = [5, 7, 9, 15, 20, 22, 25, 30, 32, 35, 37, 40, 50, 55, 60, 100, 2000]
# 2000 is a severe outlier – box shrinks visually due to scale
```

Experiment 3 — Repeated values shift the median

Adding many repeated 56s and 57s increases the density of values in the upper-middle range. The notebook comment says: "median now is a little more risen" — demonstrating that the median responds to where the bulk of data is concentrated, not just to min/max values.

```
data = [5, 7, 9, 15, 20, 22, 25, 30, 32, 35, 37, 40, 50, 55,
        56, 56, 56, 56, 56, 56, 56, 56, 56, 56, 57, 60]
# Many 56s → median rises toward the 56/57 cluster
```


Notebook 4: Visualization with Seaborn

File: VisualizationSeaborn.ipynb

Seaborn is a high-level data visualization library built on top of Matplotlib. It produces richer, more statistically informative plots with much less code. This notebook uses real Olympic athlete data (`athlete_events.csv`) to demonstrate Seaborn's core advantage: encoding multiple data dimensions — color, shape, and size — into a single plot automatically.

Data Exploration

Before plotting, the notebook inspects the dataset using standard Pandas methods:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = pd.read_csv('athlete_events.csv')

data.describe()      # statistics: mean, std, min, max per column
data.head()          # first 5 rows
data.tail()          # last 5 rows
data.columns         # list all column names
data.isnull()        # shows True/False where data is missing
data.info()          # column dtypes and non-null counts

data['Medal'].unique() # unique values: Gold, Silver, Bronze, NaN
data['Sex'].unique()   # unique values: M, F
data['Season'].unique()# unique values: Summer, Winter
```

sns.set_style() — Background Themes

Definition: Controls the background and grid appearance for all subsequent Seaborn plots. Called once before plotting — affects every plot drawn after it.

```
sns.set_style("whitegrid")  # white bg + grid lines (easiest to read)
sns.set_style("darkgrid")   # dark bg + grid lines (good for presentations)
sns.set_style("white")      # plain white, no grid (cleanest for publications)
sns.set_style("dark")       # dark background, no grid
sns.set_style("ticks")      # white background with tick marks only
```

Scatter Plot — sns.scatterplot()

Definition: Plots each DataFrame row as a point using x and y columns. Far more powerful than `plt.scatter()` because it natively encodes up to three additional variables via hue (color), style (marker shape), and size.

```
sns.set_style("white")
```

```

sns.scatterplot(
    x="Height",
    y="Weight",
    hue="Sex",      # color by sex: M=one color, F=another
    style="Medal",  # shape by medal: Gold/Silver/Bronze/NaN each differ
    size="Age",     # larger dot = older athlete
    data=data
)
plt.xlabel("Height of Athletes")
plt.ylabel("Weight of Athletes")
plt.title("Athletes Weight vs Height")
plt.show()

```

Property / Value	Description
x, y	Column names for the horizontal and vertical axes
hue	Column to color-encode — each unique value gets a different color. Notebook comment: 'used to emphasize specific things'
style	Column to shape-encode — each unique value gets a different marker shape (circle, cross, triangle, etc.)
size	Numeric column — larger values produce larger dots; smaller values produce smaller dots
data	The Pandas DataFrame containing all the named columns

Line Plot — `sns.lineplot()`

Definition: Draws a line plot and automatically computes and displays a shaded confidence interval band around the line. When hue is set, draws one line per group. This notebook plots average weight per height, split by sex.

```

sns.set_style("whitegrid")
sns.lineplot(x="Height", y="Weight", hue="Sex", data=data)
plt.xlabel("Height of Athletes")
plt.ylabel("Weight of Athletes")
plt.title("Athletes Weight vs Height")
plt.show()

```

Distribution Plot — `sns.displot()`

Definition: Plots the distribution of a single variable. Supports histogram mode (default) and KDE (Kernel Density Estimate) mode. With hue, draws overlapping distributions for each group — great for comparing male vs female height distribution in this dataset.

```

# Histogram mode (default)
sns.set_style("white")
sns.displot(x="Height", hue="Sex", data=data)
plt.ylabel("Frequency")

```

```
plt.title("Athletes Height Distribution")
plt.show()

# KDE mode – smooth continuous density curve
sns.displot(x="Height", hue="Sex", kind="kde", data=data)
# KDE = Kernel Density Estimate:
# visualizes the distribution of observations as a smooth curve
```

Property / Value	Description
kind="hist"	Histogram (default) — counts values in fixed-width bins and draws bars
kind="kde"	Kernel Density Estimate — a smooth continuous curve representing the probability distribution
kind="ecdf"	Empirical Cumulative Distribution Function — shows proportion of data up to each value

Bar Plot — sns.barplot()

Definition: Shows the mean (average) of a numeric variable for each category, with automatic confidence interval error bars. Here it displays the average height of athletes grouped by medal type and sex.

```
sns.barplot(x="Medal", y="Height", hue="Sex", data=data)
plt.title("Medals by Height")
plt.show()
# Each bar = average height for that medal-sex combination
# Error bars = 95% confidence interval
```

Categorical Plot with Facets — sns.catplot()

Definition: A figure-level function that creates separate sub-panels (facets) for each unique value of a column. The col parameter creates side-by-side panels — here one for Summer and one for Winter Olympics. Extremely powerful for visual comparison across a categorical variable.

```
sns.catplot(
    x="Medal",
    y="Height",
    hue="Sex",
    col="Season",  # one panel per season: Summer | Winter
    data=data
)
# Produces two side-by-side plots automatically
```

Property / Value	Description
col	Column to split into side-by-side panels (facet columns)

row	Column to split into stacked panels (facet rows)
kind	Plot type within each panel: 'strip' (default), 'box', 'violin', 'bar', 'point', 'swarm'

Correlation Heatmap — `sns.heatmap()`

Definition: Visualizes a matrix of numbers as a color grid — warmer colors for higher values, cooler for lower. Used here with `data.corr()` to show how all numeric columns relate to each other. Values range from -1.0 (strong negative correlation) to +1.0 (strong positive correlation).

```
# Step 1: compute correlation matrix (numbers only)
corr_matrix = data.corr(numeric_only=True)
# numeric_only=True skips string columns (Name, Sport, etc.)
# because correlation only works on numbers

# Step 2: draw the heatmap
sns.heatmap(corr_matrix, annot=True)
# annot=True → prints the numeric value inside each colored cell
```

Property / Value	Description
<code>annot=True</code>	Prints the numeric correlation value inside each colored cell
<code>annot=False</code>	Color only, no numbers (default)
<code>numeric_only=True</code>	Required on <code>.corr()</code> to skip non-numeric (string) columns
<code>cmap="coolwarm"</code>	Color palette — blue → white → red diverging scale; good for correlations
<code>cmap="viridis"</code>	Sequential purple-to-yellow scale
<code>fmt='.2f'</code>	Format string for annotation numbers (e.g. 2 decimal places)
<code>linewidths=0.5</code>	Width of lines separating cells in the grid

End of Guide